differently to what we have done with heaps in chapter 2: we'll go back to discussing this choice more in detail in section 3.3.2.

**Listing 3.1 class Treap**

```
class Node
  key
  #type double
  priority

  #type Node
  left
  #type Node
  right
  #type Node
  parent

  function Node(key, priority)
    (this.key, this.priority) ← (key, priority)
    this.left ← null
    this.right ← null

class Treap
  #type Node
  root

  function Treap()
    root ← new Node()
```

In this implementation, the `Treap` class is mostly a wrapper for the root of the actual tree; each node of the tree holds two attributes for a `key` (that can be of any type, as long as there is a total ordering defined on the possible values) and a `priority`, that here we'll assume to be a double precision number (an integer or any type with a total ordering could work too, but we'll see in the next section that double works better).